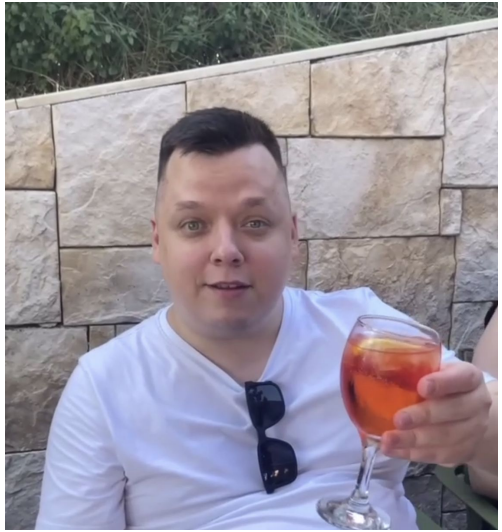
На связи Олег

- Выпускник школы программистов в 2018 году
- Тимлид команды Talantinos
- Будете слушать меня 4 лекции

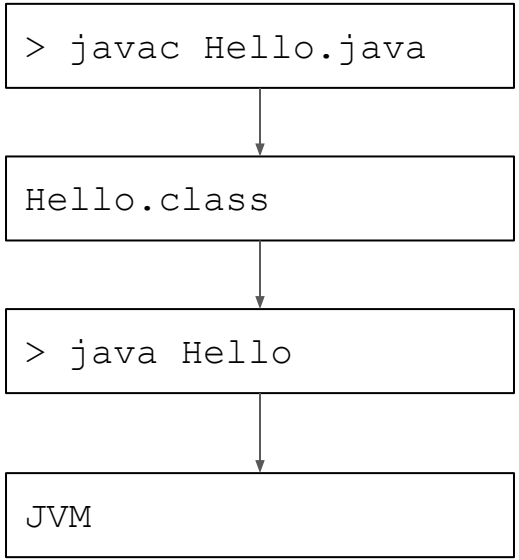


Основы Java

(совсем основы)

Особенности

- Код компилируется в байт-код, выполняемый виртуальной машиной JVM, тем самым достигается кроссплатформенность



Особенности

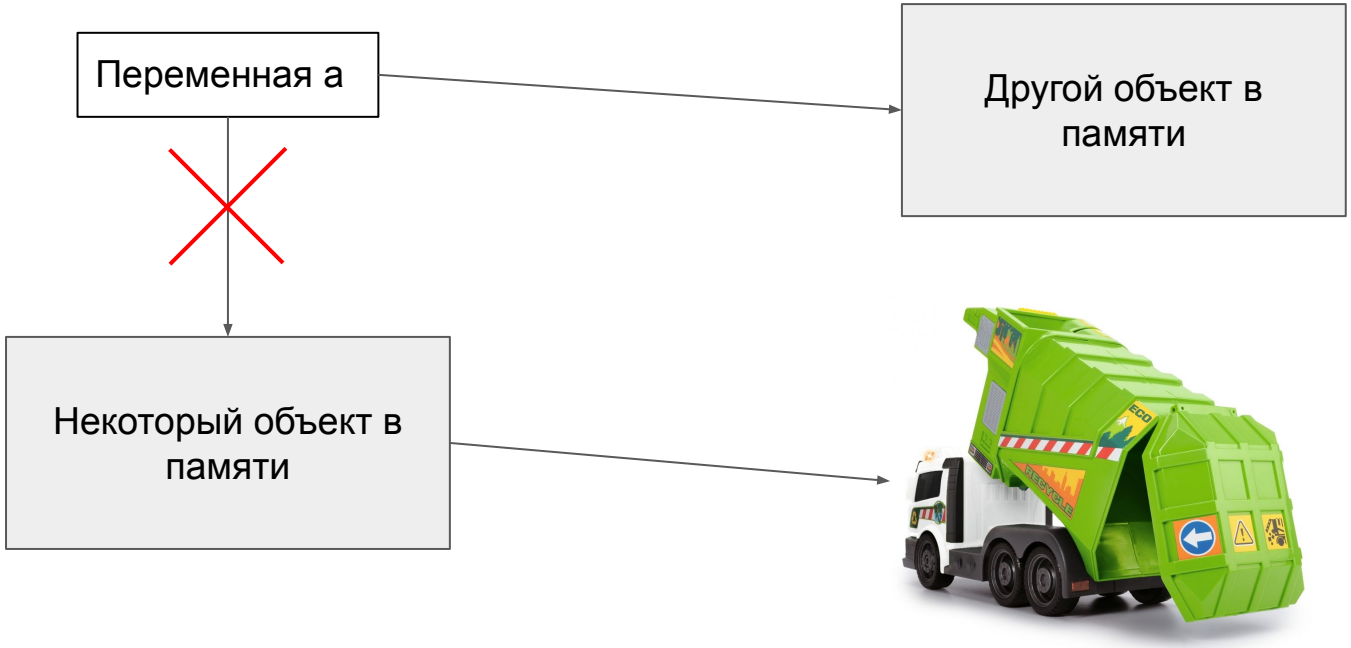
- Объектно-ориентированный, все есть объект, глобального кода не существует

```
public class Program {

    public static void main (String args[]) {
        System.out.println("Hello Java!");
    }
}
```

Особенности

- Есть сборщик мусора, им удаляются объекты без ссылок



Особенности

- Есть сборщик мусора, им удаляются объекты без ссылок

Особенности для любителей C++

- Отсутствует множественное наследование
- Отсутствуют операции с указателями

Структура программы

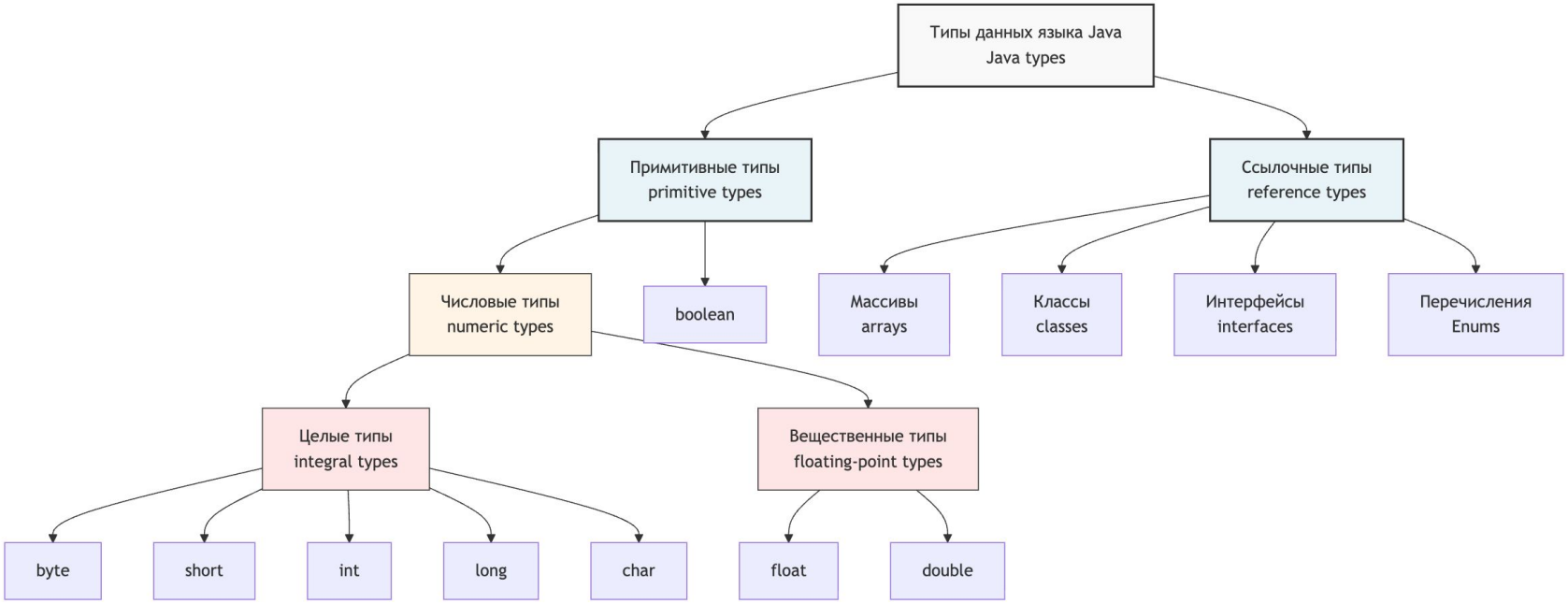
```java
public class Program {

    public static void main (String args[]) {
        System.out.println("Hello Java!");
    }
}
```

static - метод принадлежит классу, а не объекту этого класса

```
> java Program someArg1 someArg2
```

Типы


Типы данных языка Java
Java types
Примитивные типы
primitive types
Ссылочные типы
reference types
Числовые типы
numeric types
boolean
Массивы
arrays
Классы
classes
Интерфейсы
interfaces
Перечисления
Enums
Целые типы
integral types
Вещественные типы
floating-point types
byte
short
int
long
char
float
double

Примитивные типы

Целочисленные типы

Тип	Размер (бит)	Диапазон
`byte`	8 бит	от -128 до 127
`short`	16 бит	от -32768 до 32767
`char`	16 бит	беззнаковое целое число, представляющее собой символ UTF-16 (буквы и цифры)
`int`	32 бит	от -2147483648 до 2147483647
`long`	64 бит	от -9223372036854775808L до 9223372036854775807L

Целочисленные типы

```java
public class IntegralTypes {
    public static void main(String[] args) {
        byte b = 216; // Вот тут будет ошибка, т.к. у нас диапазон от -128 до 127!
        short s = 1123;
        int i = 64536;
        long l = 2147483648L; // Постфикс l или L обозначает литералы типа long
        System.out.println(i);
        System.out.println(b);
        System.out.println(s);
        System.out.println(l);
    }
}
```

Целочисленные типы

```java
public class Characters {
    public static void main(String[] args) {
        char a = 'a', b, c = 'c';
        b = (char) ((a + c) / 2); // Можно складывать, вычитать, делить и умножать
        // Но из-за особенностей арифметики Java результат приходится приводить к типу char явно
        System.out.println(b); // Выведет символ 'b'
    }
}
```

Вещественные типы

Тип	Размер (бит)	Диапазон
`float`	32	от 1.4e-45f до 3.4e+38f
`double`	64	от 4.9e-324 до 1.7e+308

```java
public class FloatingPointTypes {
    public static void main(String[] args) {
        double a, b = 4.12;
        a = 22.1 + b;
        float pi = 3.14f; // При использовании типа float требуется указывать суффикс f или F
        // так как без них типом литерала будет считаться double
        float anotherPi = (float) 3.14; // Можно привести явно
        double c = 27;
        double d = pi * c;
        System.out.println(d);
    }
}
```

Ссылочные типы

Строки

Они особенные, операция сложения переопределена

```java
public class Strings {
    public static void main(String[] args) {
        String a = "Hello", b = "World";
        System.out.println(a + " " + b); // Здесь + означает объединение (конкатенацию) строк

        // Строки конкатенируются слева направо, надо помнить это когда соединяешь строку и примитив
        String c = 2 + 2 + ""; // "4"
        String d = "" + 2 + 2; // "22"
        d = "" + (2 + 2); // а теперь d тоже "4"
    }
}
```

Строки

В JVM есть пул строк

```
public class String {
  public static void main(String[] args) {
    String foo = "a string";
    String bar = "a string"; // bar будет указывать на тот же объект что и foo

    // Чтобы гарантированно создать новую строку надо вызвать конструктор
    String baz = new String("a string");

    assert(foo == bar); // Ссылки равны, из-за кэша строк
    assert(foo.equals(bar)); // Содержимое одинаково
    assert(foo != baz); // Явно создали другой объект, ссылки разные
    assert(foo.equals(baz)); // Но содержимое все так же одинаково
  }
}
```

Массивы

```
int nums[];
int[] nums2;

nums = new int[4];
nums2 = { 1, 2, 3, 4 };

System.out.println(nums2.length);

int[][] nums2 = { { 0, 1, 2 }, { 3, 4, 5 } };

System.out.println(nums2[0][1]);
```

Классы

```java
public class Candidate {
  private Integer id;
  private String name;

  // конструктор по умолчанию
  public Candidate() {}

  // конструктор с параметрами
  public Candidate(Integer id, String name) {
    this.id = id;
    this.name = name;
  }

  public Integer getId() { return id; }
  public void setId(Integer id) { this.id = id; }

  public String getName() { return name; }
  public void setName(String name) { this.name = name; }
}
```

По умолчанию поля равны null

```java
Candidate oleg = new Candidate(1, "Oleg");

Candidate noname = new Candidate();
assert(noname.getName() == null)

Candidate oleg2 = oleg;
assert(oleg == oleg2);
assert(oleg.equals(oleg2)); // как работает?
```

По умолчанию equals() сравнивает объекты по ссылке - для правильной проверки содержимого нужно писать свою реализацию

Модификаторы доступа

public - доступны всем

private - доступны только данному классу

protected - доступны данному классу и его наследникам

без модификатора - package-private или модификатор по умолчанию - доступны всем в рамках текущего пакета

Что за пакеты?

```java
package ru.hh.tms.candidate;

import java.time.Instant;
import ru.hh.tms.Contact;

public class Candidate {
    private Integer id;
    private Instant createdAt;
    private Contact[] contacts;
    ...
}
```

Можно задать имя пакета (совпадает с вложенностью папок в которой лежит класс)

Вот так импортируются другие классы, чтобы их можно было использовать (наши, из библиотек, или из JDK)

Обертки над примитивами

Существуют встроенные типы Integer, Boolean, Long и т.д.

```
Integer a = null;
a = 2;
int b = a.intValue(); // Распаковка (unboxing) в примитив
int c = a; // Автоматическая расспаковка

Integer d; // не инициализирована, если заиспользовать,
программа не скомпилируется
```

Бьем рекорды

```
public record Candidate(Integer id, String name) {

}


Candidate oleg = new Candidate(1, "Olegator");

// getters
oleg.id();
oleg.name();

// fields final

// has equals and hashCode
assert(oleg.equals(new Candidate(1, "Olegator")))
```

ВОПРОСЫ

ПЕРЕРЫВ

Наследование

```java
public class Animal {
  protected int age;
  public void eat() { println("I'm eating"); }
}

public Dog extends Animal {
  @Override
  public void eat() {
    gavgav();
    super.eat();
  }

  public void gavgav() {
    if (age > 5) {
      println("gav");
    } else {
      println("rrrr, GAV GAV GAV");
    }
  }
}
```

```java
Dog timofei = new Dog();
Animal sharik = new Dog();

Animal timofei2 = timofei;
Object sharik2 = sharik;
```

Все объекты неявно наследуются от класса Object

Наследование

```java
public class Animal {
  protected int age;
  public void eat() { println("I'm eating"); }
}

public Dog extends Animal {
  @Override
  public void eat() {
    gavgav();
    super.eat();
  }

  public void gavgav() {
    if (age > 5) {
      println("gav");
    } else {
      println("rrrr, GAV GAV GAV");
    }
  }
}
```

```java
public void someMethod(Animal animal) {
  animal.eat(); // Какой eat вызовется?

  if (animal instanceof Dog) {
    ((Dog) animal).gavgav();
  }
}
```

Наследование

```java
public class Animal {
  protected int age;
  public void eat() { println("I'm eating"); }
}

public Dog extends Animal {
  @Override
  public void eat() {
    gavgav();
    super.eat();
  }

  public void gavgav() {
    if (age > 5) {
      println("gav");
    } else {
      println("rrrr, GAV GAV GAV");
    }
  }
}
```

```java
public void someMethod(Animal animal) {
  animal.eat();

  if (animal instanceof Dog dog) {
    dog.gavgav();
  }
}
```

Абстрактные классы

```
public abstract class Animal {
  protected int age;
  public void eat() { println("I'm eating"); }
  public abstract void sleep();
}

public Cat extends Animal {
  @Override
  public void sleep() {
    println("Sleeping cat");
  }
}
```

Пример equals

```java
class Candidate {
  public Integer id;
  public String name;

  @Override
  public boolean equals(Object o) {
    if (this == o) {
      return true;
    }
    if (o == null || getClass() != o.getClass()) {
      return false;
    }
    return Objects.equals(id, o.id) &&
      Objects.equals(name, o.name);
  }
}
```

Перечисление enum

```java
public static final String ACTIVE = "active";
public static final String ARCHIVE = "archive";

public class Vacancy {
  private String status;
  ...
}

vacancy.setStatus(ACTIVE);
if (ACTIVE.equals(vacancy.getStatus())) {
  doSomething();
}
```

```java
public enum VacancyStatus {
  ACTIVE,
  ARCHIVE
}

public class Vacancy {
  private VacancyStatus status;
  ...
}

if (vacancy.getStatus() == VacancyStatus.ACTIVE) {
  doSomething();
}
```

Дженерики

```java
public class Container<T> {
  private T object;

  public Container(T object) {
    this.object = object;
  }

  public T getObject() {
    return object;
  }

  public Container<T> setObject(T object) {
    this.object = object;
    return this;
  }
}
```

diamond operator

```java
Container<Integer> i = new Container<>(1);
i.setObject(2);

Container<String> s = new Container<>("absd");
i.setObject("abcd");
```

Не очень умный List

```java
public class List<E> {
  private Object[] data;
  private int size = 0;

  public List(int maxSize) {
    data = new Object[maxSize];
  }

  public void add(E element) {
    data[size] = element;
    size++;
  }

  public E get(int index) {
    return (E) data[index];
  }
}
```

так написать можно

а так нельзя

```java
public class List<E> {
  private E[] data;
  ...

  public List(int maxSize) {
    data = new E[maxSize];
  }

  ...
}
```

Дженерики

Из наследования параметра дженерика не следует наследования самого дженерика

```
Container<Dog> dog = new Container<>(new Dog());
Container<Animal> animal = dog; // ОШИБКА!
```

Дженерики

```java
public class List<E> {
  private Object[] data;
  private int size = 0;

  public List(int maxSize) {
    data = new Object[maxSize];
  }

  public void add(E element) {
    data[size] = element;
    size++;
  }

  public E get(int index) {
    return (E) data[index];
  }
}
```

```java
List<Dog> dogs = new List<>(10);

List<Animal> animals = dogs;

animals.add(new Cat());

Dog firsDog = dogs.get(0); // Dog or Cat
```

Создаем список собак

Дженерики

```java
public class List<E> {
  private Object[] data;
  private int size = 0;

  public List(int maxSize) {
    data = new Object[maxSize];
  }

  public void add(E element) {
    data[size] = element;
    size++;
  }

  public E get(int index) {
    return (E) data[index];
  }
}
```

```java
List<Dog> dogs = new List<>(10);

List<Animal> animals = dogs;

animals.add(new Cat());

Dog firsDog = dogs.get(0); // Dog or Cat
```

Список собак - это список животных

Дженерики

```java
public class List<E> {
  private Object[] data;
  private int size = 0;

  public List(int maxSize) {
    data = new Object[maxSize];
  }

  public void add(E element) {
    data[size] = element;
    size++;
  }

  public E get(int index) {
    return (E) data[index];
  }
}
```

```java
List<Dog> dogs = new List<>(10);

List<Animal> animals = dogs;

animals.add(new Cat());

Dog firsDog = dogs.get(0); // Dog or Cat
```

В список животных добавляем кошку, кошка ведь животное

Дженерики

```java
public class List<E> {
  private Object[] data;
  private int size = 0;

  public List(int maxSize) {
    data = new Object[maxSize];
  }

  public void add(E element) {
    data[size] = element;
    size++;
  }

  public E get(int index) {
    return (E) data[index];
  }
}
```

```java
List<Dog> dogs = new List<>(10);

List<Animal> animals = dogs;

animals.add(new Cat());

Dog firsDog = dogs.get(0); // Dog or Cat
```

Изначальный dogs ссылается на этот же список, но первый элемент оказывается ~~с сюрпризом~~ кошкой

Дженерики

Как же покормить любых животных?

```
public void feed(List<? extends Animal> animals) {
  for(Animal animal : animals) {
    animal.eat();
  }
}
```

Дженерики

```java
// Можно сделать дженерный метод
public static <T> void method(T t) {
  System.out.println("Param: " + t.toString());
}
```

```java
// Можно давать ограничения
public static <T extends Animal> T someMethod(Container<T> animalContainer) {
  ...
  return animalCointainger.get();
}
```

```java
// Ограничений, как и типов, может быть несколько
public static <T, E, R extends SomeClass & SomeInterface> T method(E e, R r) {
  ...
}
```

ВОПРОСЫ

ПЕРЕРЫВ

Интерфейсы

```java
public class App {

  interface Error {
    String description();
  }

  public class NotFoundError implements Error {
    private String id;
    @Override
    public String description() {
      return "Entity with id " + id + " not found";
    }
    // ... getter and setter ...
  }

  public static void printError(Error error) {
    System.out.println(error.description());
  }
}
```

```java
NotFoundError error = new NotFoundError();
error.setId(1);
App.printError(error);
```

```java
App.printError(new Error() {
  public String description() {
    return "Так тоже можно делать";
  }
});
```

Интерфейс Comparable

```java
public interface Comparable<T> {
  int compareTo(T item);
}
```

```java
public class Candidate implements Comparable<Candidate> {
  private String firstName;
  private String lastName;

  @Override
  public int compareTo(Candidate candidate) {
    int lastNameComp = lastName.compareTo(candidate.lastName);
    return lastNameComp == 0 ? firstName.compareTo(candidate.firstName) : lastNameComp;
  }
}
```

Интерфейс Comparable

```
public interface Comparable<T> {
  int compareTo(T item);
}
```

```
public static <T extends Comparable<T>> T max(T first, T second) {
  return first.compareTo(second) > 0 ? first : second;
}

Candidate max = max(Oleg, Vasya);
```

Интерфейсы функциональные

Как бы передать в метод функцию/метод, а не объект…
Ну наверное можно создать интерфейс, содержащий всего одну эту функцию/метод...

```
public interface Action {
  void action();
}

public static void executeAction(Action action) {
  System.out.println("Выполняю метод Action");
  action.action();
}
```

И каждый раз создавать просто анонимный экземпляр, чтобы передать эту самую функцию/метод...

```
executeAction(new Action() {
  @Override
  public void action() {
    System.out.println("Мое действие!!");
  }
});
```

Интерфейсы функциональные

Как бы передать в метод функцию/метод, а не объект…
Ну наверное можно создать интерфейс, содержащий всего одну эту функцию/метод...

```
public interface Action {
  void action();
}

public static void executeAction(Action action) {
  System.out.println("Выполняю метод Action");
  action.action();
}
```

Или использовать фичи java

```
executeAction(() -> System.out.println("Мое действие!!"));
```

Интерфейсы функциональные

```
@FunctionalInterface
interface Action {
  void action();
}
```

Интерфейсы функциональные

```java
void foo(Runnable runnable) {
  runnable.run();
}

void bar(Consumer<String> consumer) {
  consumer.accept("Строка");
}

void baz(Supplier<String> supplier) {
  System.out.println(supplier.get());
}

void fooBarBaz(Function<Integer, String> personNameGetter) {
  System.out.println("Name - " + personNameGetter.apply(1));
}
```

```java
foo(() -> println("Make program, not war"));

bar(System.out::println);

baz(() -> "Чья-то строка");

fooBarBaz(id -> "PersonName" + id);
```

За кадром

1. Исключения и работа с ними

За кадром

1. Исключения и работа с ними
2. Optional

За кадром

1. Исключения и работа с ними
2. Optional
3. Любая фича, про которую вы вспомните и скажите “А вы не рассказали про это!!”

ВОПРОСЫ

Литература и ссылки

- https://metanit.com/java/tutorial
- Герберт Шилдт. Java. Полное руководство

СПАСИБО ЗА ВНИМАНИЕ!